Ex. No: 08	DATA PERSISTENCE USING SQLITE AND FIREBASE
Date of Exercise	08.09.2026
Git-hub Host Link	https://santhosh444-ai.github.io/FLUTTER-EXP-8/

Aim:

To Implement data persistence in a Flutter application using SQLite for local storage and Firebase Cloud Firestore for cloud storage, to perform basic data operations such as insertion, retrieval, updating, and deletion.

Description:

This experiment demonstrates how Flutter applications can store data permanently using two different approaches. **SQLite** stores student information locally on the device, while **Firebase Cloud Firestore** stores student information in the cloud. The application uses students collection containing **ID, name, and course** details and displays the stored records in the Flutter interface.

Procedure:

Step 1 – Create the SQLite Flutter project using flutter create sqlite_app.

Step 2: Add the sqflite and path packages to pubspec.yaml and run flutter pub get.

Step 3: Create database_helper.dart inside the lib folder.

Step 4: Create the SQLite students table with fields id, name, and course.

Step 5: Implement SQLite CRUD operations: **Create, Read, Update, and Delete**.

Step 6: Create text fields in main.dart to enter student name and course, insert the data into SQLite.

Step 7: Retrieve SQLite records and display them using ListView.builder().

Step 8: Close and restart the application to verify that the SQLite data remains stored.

Step 9: Create a Firebase project using the Firebase Console and enable **Cloud Firestore**.

Step 10: Create a new Flutter project and connect it to Firebase using flutterfire configure.

Step 11: Add the firebase_core and cloud_firestore packages using flutter pub add.

Step 12: Initialize Firebase in main.dart using Firebase.initializeApp().

Step 13: Create a Firestore collection named students with fields such as name and course.

Step 14: Insert student information into the Firebase students collection.

Step 15: Retrieve Firebase records and display them in the Flutter application.

Program:

```
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'database_helper.dart';
import 'firebase_options.dart';
Future<void> main() async {
WidgetsFlutterBinding.ensureInitialized();
initializeSQLite();
await Firebase.initializeApp(
options: DefaultFirebaseOptions.currentPlatform,
);
runApp(const StudentDataApp());
}
class StudentDataApp extends StatelessWidget {
const StudentDataApp({super.key});
@override
Widget build(BuildContext context) {
return MaterialApp(
debugShowCheckedModeBanner: false,
title: 'Student Data Persistence',
```

```
theme: ThemeData(
colorScheme: ColorScheme.fromSeed(
seedColor: Colors.indigo,
),
useMaterial3: true,
),
home: const StudentHomePage(),
);
}
}
class StudentHomePage extends StatefulWidget {
const StudentHomePage({super.key});
@override
State<StudentHomePage> createState() => _StudentHomePageState();
}
class _StudentHomePageState extends State<StudentHomePage> {
final nameController = TextEditingController();
final courseController = TextEditingController();
List<Map<String, dynamic>> sqliteStudents = [];
List<Map<String, dynamic>> firebaseStudents = [];
bool loadingSQLite = false;
bool loadingFirebase = false;
```

```
@override
void initState() {
super.initState();
loadSQLiteStudents();
loadFirebaseStudents();
}
// ============================================================
// SQLITE
// ============================================================
Future<void> loadSQLiteStudents() async {
setState(() {
loadingSQLite = true;
});
try {
final data = await DatabaseHelper.instance.getStudents();
if (!mounted) return;
setState(() {
sqliteStudents = data;
loadingSQLite = false;
});
} catch (e) {
if (!mounted) return;
```

```
setState(() {
loadingSQLite = false;
});
showMessage('SQLite error: $e');
}
}
Future<void> addSQLiteStudent(
String name,
String course,
) async {
await DatabaseHelper.instance.insertStudent(
name,
course,
);
await loadSQLiteStudents();
}
Future<void> updateSQLiteStudent(
int id,
String name,
String course,
) async {
await DatabaseHelper.instance.updateStudent(
```

```
id,
name,
course,
);
await loadSQLiteStudents();
}
Future<void> deleteSQLiteStudent(int id) async {
await DatabaseHelper.instance.deleteStudent(id);
await loadSQLiteStudents();
}
// ============================================================
// FIREBASE
// ============================================================
Future<void> loadFirebaseStudents() async {
setState(() {
loadingFirebase = true;
});
try {
final snapshot = await FirebaseFirestore.instance
.collection('students')
.orderBy('name')
.get();
```

```
final data = snapshot.docs.map((doc) {
final student = doc.data();
return {
'id': doc.id,
'name': student['name'] ?? '',
'course': student['course'] ?? '',
};
}).toList();
if (!mounted) return;
setState(() {
firebaseStudents = data;
loadingFirebase = false;
});
} catch (e) {
if (!mounted) return;
setState(() {
loadingFirebase = false;
});
showMessage('Firebase error: $e');
}
}
Future<void> addFirebaseStudent(
```

```
String name,
String course,
) async {
await FirebaseFirestore.instance
.collection('students')
.add({
'name': name,
'course': course,
'createdAt': FieldValue.serverTimestamp(),
});
await loadFirebaseStudents();
}
Future<void> updateFirebaseStudent(
String id,
String name,
String course,
) async {
await FirebaseFirestore.instance
.collection('students')
.doc(id)
.update({
'name': name,
```

```
'course': course,
'updatedAt': FieldValue.serverTimestamp(),
});
await loadFirebaseStudents();
}
Future<void> deleteFirebaseStudent(String id) async {
await FirebaseFirestore.instance
.collection('students')
.doc(id)
.delete();
await loadFirebaseStudents();
}
// ============================================================
// ADD TO BOTH
// ============================================================
Future<void> addStudent() async {
final name = nameController.text.trim();
final course = courseController.text.trim();
if (name.isEmpty || course.isEmpty) {
showMessage(
'Please enter student name and course',
);
```

```
return;
}
try {
await addSQLiteStudent(name, course);
await addFirebaseStudent(name, course);
nameController.clear();
courseController.clear();
showMessage(
'Student saved to SQLite and Firebase',
);
} catch (e) {
showMessage('Error: $e');
}
}
// ============================================================
// EDIT DIALOG
// ============================================================
Future<void> showEditSQLiteDialog(
Map<String, dynamic> student,
) async {
final name = TextEditingController(
text: student['name'],
```

```
);
final course = TextEditingController(
text: student['course'],
);
await showDialog(
context: context,
builder: (context) {
return AlertDialog(
title: const Text('Edit SQLite Student'),
content: Column(
mainAxisSize: MainAxisSize.min,
children: [
TextField(
controller: name,
decoration: const InputDecoration(
labelText: 'Name',
),
),
const SizedBox(height: 12),
TextField(
controller: course,
decoration: const InputDecoration(
```

```
labelText: 'Course',
),
),
],
),
actions: [
TextButton(
onPressed: () {
Navigator.pop(context);
},
child: const Text('Cancel'),
),
ElevatedButton(
onPressed: () async {
if (name.text.trim().isEmpty ||
course.text.trim().isEmpty) {
return;
}
await updateSQLiteStudent(
student['id'],
name.text.trim(),
course.text.trim(),
```

```
);
if (context.mounted) {
Navigator.pop(context);
}
},
child: const Text('Update'),
),
],
);
},
);
}
Future<void> showEditFirebaseDialog(
Map<String, dynamic> student,
) async {
final name = TextEditingController(
text: student['name'],
);
final course = TextEditingController(
text: student['course'],
);
await showDialog(
```

```
context: context,
builder: (context) {
return AlertDialog(
title: const Text('Edit Firebase Student'),
content: Column(
mainAxisSize: MainAxisSize.min,
children: [
TextField(
controller: name,
decoration: const InputDecoration(
labelText: 'Name',
),
),
const SizedBox(height: 12),
TextField(
controller: course,
decoration: const InputDecoration(
labelText: 'Course',
),
),
],
),
actions: [
```

```
TextButton(
onPressed: () {
Navigator.pop(context);
},
child: const Text('Cancel'),
),
ElevatedButton(
onPressed: () async {
if (name.text.trim().isEmpty ||
course.text.trim().isEmpty) {
return;
}
await updateFirebaseStudent(
student['id'],
name.text.trim(),
course.text.trim(),
);
if (context.mounted) {
Navigator.pop(context);
}
},
child: const Text('Update'),
```

```
),
],
);
},
);
}
// ============================================================
// MESSAGE
// ============================================================
void showMessage(String message) {
if (!mounted) return;
ScaffoldMessenger.of(context).showSnackBar(
SnackBar(
content: Text(message),
),
);
}
// ============================================================
// UI
// ============================================================
@override
Widget build(BuildContext context) {
```

```
return Scaffold(
appBar: AppBar(
title: const Text(
'Student Data Persistence',
),
centerTitle: true,
),
body: SingleChildScrollView(
padding: const EdgeInsets.all(16),
child: Column(
crossAxisAlignment: CrossAxisAlignment.stretch,
children: [
const Text(
'SQLite + Firebase',
textAlign: TextAlign.center,
style: TextStyle(
fontSize: 28,
fontWeight: FontWeight.bold,
),
),
const SizedBox(height: 6),
Text(
```

```
kIsWeb
? 'Web • SQLite WASM + Firebase'
: 'Desktop • SQLite + Firebase',
textAlign: TextAlign.center,
style: TextStyle(
color: Colors.grey.shade700,
),
),
const SizedBox(height: 25),
// INPUT CARD
Card(
child: Padding(
padding: const EdgeInsets.all(16),
child: Column(
children: [
const Text(
'Add Student',
style: TextStyle(
fontSize: 20,
fontWeight: FontWeight.bold,
),
),
```

```
const SizedBox(height: 15),
TextField(
controller: nameController,
decoration: const InputDecoration(
labelText: 'Student Name',
prefixIcon: Icon(Icons.person),
border: OutlineInputBorder(),
),
),
const SizedBox(height: 12),
TextField(
controller: courseController,
decoration: const InputDecoration(
labelText: 'Course',
prefixIcon: Icon(Icons.school),
border: OutlineInputBorder(),
),
),
const SizedBox(height: 15),
SizedBox(
width: double.infinity,
child: ElevatedButton.icon(
```

```
onPressed: addStudent,
icon: const Icon(Icons.add),
label: const Text(
'Save to SQLite + Firebase',
),
),
),
],
),
),
),
const SizedBox(height: 25),
// SQLITE
const Text(
'SQLite - Local Storage',
style: TextStyle(
fontSize: 22,
fontWeight: FontWeight.bold,
),
),
const SizedBox(height: 10),
if (loadingSQLite)
```

```
const Center(
child: CircularProgressIndicator(),
)
else if (sqliteStudents.isEmpty)
const Card(
child: Padding(
padding: EdgeInsets.all(16),
child: Text(
'No SQLite students found.',
),
),
)
else
...sqliteStudents.map(
(student) {
return Card(
child: ListTile(
leading: CircleAvatar(
child: Text(
student['id'].toString(),
),
),
```

```
title: Text(
student['name'],
),
subtitle: Text(
student['course'],
),
trailing: Row(
mainAxisSize: MainAxisSize.min,
children: [
IconButton(
tooltip: 'Edit',
icon: const Icon(
Icons.edit,
),
onPressed: () {
showEditSQLiteDialog(
student,
);
},
),
IconButton(
tooltip: 'Delete',
```

```
icon: const Icon(
Icons.delete,
),
onPressed: () {
deleteSQLiteStudent(
student['id'],
);
},
),
],
),
),
);
},
),
const SizedBox(height: 15),
OutlinedButton.icon(
onPressed: loadSQLiteStudents,
icon: const Icon(
Icons.refresh,
),
label: const Text(
```

```
'Refresh SQLite',
),
),
const SizedBox(height: 25),
const Divider(),
const SizedBox(height: 25),
// FIREBASE
const Text(
'Firebase - Cloud Storage',
style: TextStyle(
fontSize: 22,
fontWeight: FontWeight.bold,
),
),
const SizedBox(height: 10),
if (loadingFirebase)
const Center(
child: CircularProgressIndicator(),
)
else if (firebaseStudents.isEmpty)
const Card(
child: Padding(
```

```
padding: EdgeInsets.all(16),
child: Text(
'No Firebase students found.',
),
),
)
else
...firebaseStudents.map(
(student) {
return Card(
child: ListTile(
leading: const CircleAvatar(
child: Icon(
Icons.cloud,
),
),
title: Text(
student['name'],
),
subtitle: Text(
student['course'],
),
```

```
trailing: Row(
mainAxisSize: MainAxisSize.min,
children: [
IconButton(
tooltip: 'Edit',
icon: const Icon(
Icons.edit,
),
onPressed: () {
showEditFirebaseDialog(
student,
);
},
),
IconButton(
tooltip: 'Delete',
icon: const Icon(
Icons.delete,
),
onPressed: () {
deleteFirebaseStudent(
student['id'],
```

```
);
},
),
],
),
),
);
},
),
const SizedBox(height: 15),
OutlinedButton.icon(
onPressed: loadFirebaseStudents,
icon: const Icon(
Icons.refresh,
),
label: const Text(
'Refresh Firebase',
),
),
const SizedBox(height: 30),
Card(
child: Padding(
```

```
padding: const EdgeInsets.all(16),
child: Column(
children: [
const Text(
'Data Persistence',
style: TextStyle(
fontSize: 20,
fontWeight: FontWeight.bold,
),
),
const SizedBox(height: 10),
const Text(
'SQLite stores data locally while '
'Firebase Firestore stores data in the cloud.',
textAlign: TextAlign.center,
),
const SizedBox(height: 10),
Text(
kIsWeb
? 'Web: SQLite uses browser storage.'
: 'Windows: SQLite uses a local database file.',
textAlign: TextAlign.center,
```

```
),
],
),
),
),
],
),
),
);
}
@override
void dispose() {
nameController.dispose();
courseController.dispose();
super.dispose();
}
}
```

```
import 'package:flutter/foundation.dart';
import 'package:path/path.dart';
import 'package:sqflite_common_ffi/sqflite_ffi.dart';
import 'package:sqflite_common_ffi_web/sqflite_ffi_web.dart';

class DatabaseHelper {
static final DatabaseHelper instance =
DatabaseHelper._privateConstructor();
static Database? _database;
```

```
DatabaseHelper._privateConstructor();

Future<Database> get database async {
if (_database != null) {
return _database!;
}

_database = await _initDatabase();
return _database!;
}

Future<Database> _initDatabase() async {
if (kIsWeb) {
databaseFactory = databaseFactoryFfiWeb;

return await openDatabase(
'students_web.db',
version: 1,
onCreate: (db, version) async {
await db.execute('''
CREATE TABLE students (
id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT NOT NULL,
course TEXT NOT NULL
)
''');
},
);
}

final path = join(
await getDatabasesPath(),
'students.db',
);

return await openDatabase(
path,
version: 1,
onCreate: (db, version) async {
await db.execute('''
CREATE TABLE students (
id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
name TEXT NOT NULL,

course TEXT NOT NULL
)
''');
},
);
}
Future<int> insertStudent(
String name,
String course,
) async {
final db = await database;

return await db.insert(
'students',
{
'name': name,
'course': course,
},
);
}

Future<List<Map<String, dynamic>>> getStudents() async {
final db = await database;

return await db.query(
'students',
orderBy: 'id DESC',
);
}
Future<int> updateStudent(
int id,
String name,
String course,
) async {
final db = await database;
return await db.update(
'students',
{
'name': name,
'course': course,
},
```

```
where: 'id = ?',


whereArgs: [id],
);
}

Future<int> deleteStudent(int id) async {
final db = await database;

return await db.delete(
'students',
where: 'id = ?',
whereArgs: [id],
);
}
}

void initializeSQLite() {
if (kIsWeb) {
databaseFactory = databaseFactoryFfiWeb;
} else {
sqfliteFfiInit();
databaseFactory = databaseFactoryFfi;
}
}

// File generated by FlutterFire CLI.
// ignore_for_file: type=lint
import 'package:firebase_core/firebase_core.dart' show FirebaseOptions;
import 'package:flutter/foundation.dart'
show defaultTargetPlatform, kIsWeb, TargetPlatform;
/// Default [FirebaseOptions] for use with your Firebase apps.
///
/// Example:
/// ```dart
/// import 'firebase_options.dart';
/// // ...
/// await Firebase.initializeApp(
/// options: DefaultFirebaseOptions.currentPlatform,
/// );
/// ```
class DefaultFirebaseOptions {
```

```
static FirebaseOptions get currentPlatform {


if (kIsWeb) {
return web;
}
switch (defaultTargetPlatform) {
case TargetPlatform.android:
return android;
case TargetPlatform.iOS:
return ios;
case TargetPlatform.macOS:
return macos;
case TargetPlatform.windows:
return windows;
case TargetPlatform.linux:
throw UnsupportedError(
'DefaultFirebaseOptions have not been configured for linux - '
'you can reconfigure this by running the FlutterFire CLI again.',
);
default:
throw UnsupportedError(
'DefaultFirebaseOptions are not supported for this platform.',
);
}
}

static const FirebaseOptions web = FirebaseOptions(
apiKey: 'AIzaSyD2_SqRHdwOFdhzLDy5cbXKFpjdPgGuhHg',
appId: '1:291139208558:web:3ce393b0c2f589fbafed7f',
messagingSenderId: '291139208558',
projectId: 'student-firebase-app-b68fd',
authDomain: 'student-firebase-app-b68fd.firebaseapp.com',
storageBucket: 'student-firebase-app-b68fd.firebasestorage.app',
measurementId: 'G-HP1425X0DQ',
);

static const FirebaseOptions android = FirebaseOptions(
apiKey: 'AIzaSyAMHi3_F9LRqqkHCU2wPsrFKMDcrwfAeLA',
appId: '1:291139208558:android:4103cdb00632d5baafed7f',
messagingSenderId: '291139208558',
projectId: 'student-firebase-app-b68fd',
```

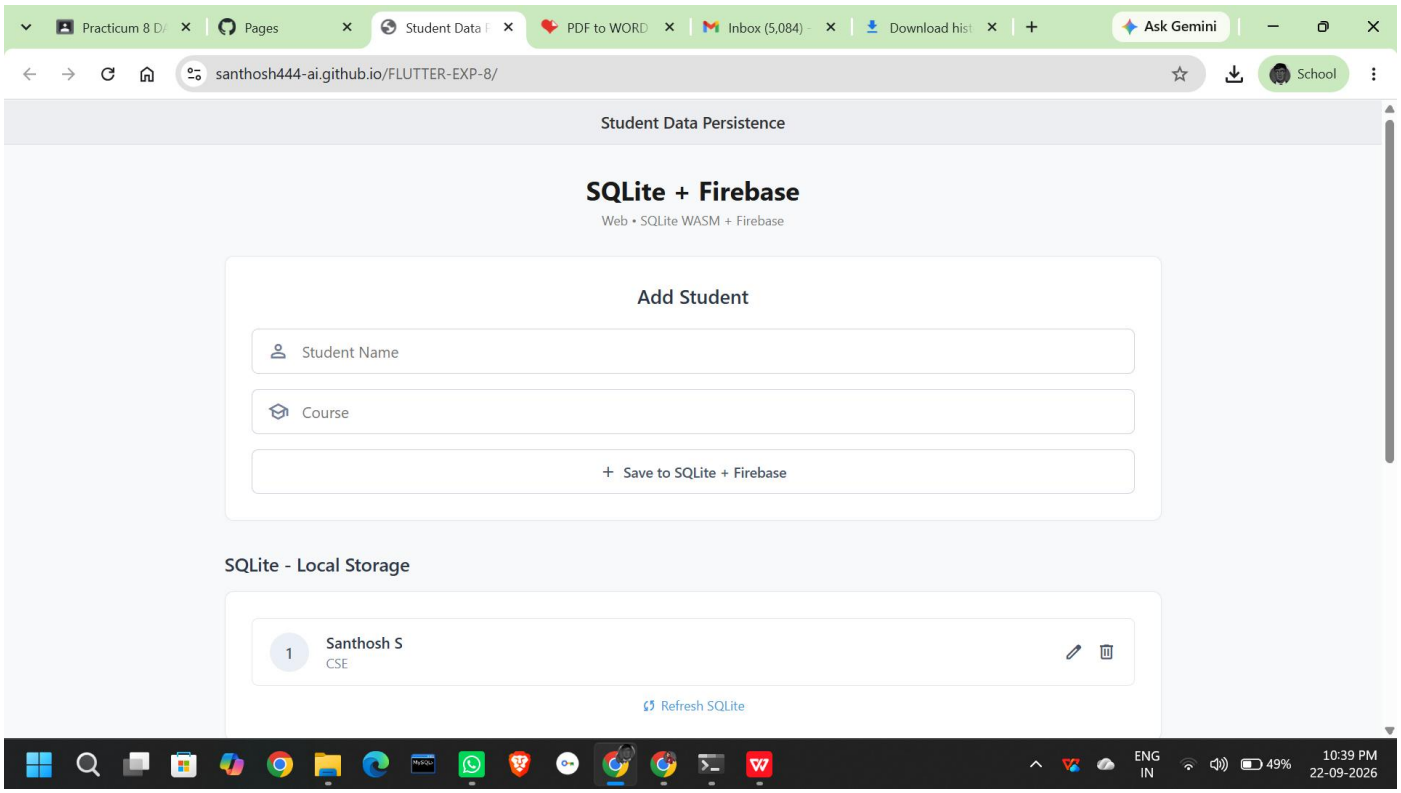
```
storageBucket: 'student-firebase-app-b68fd.firebasestorage.app',
);

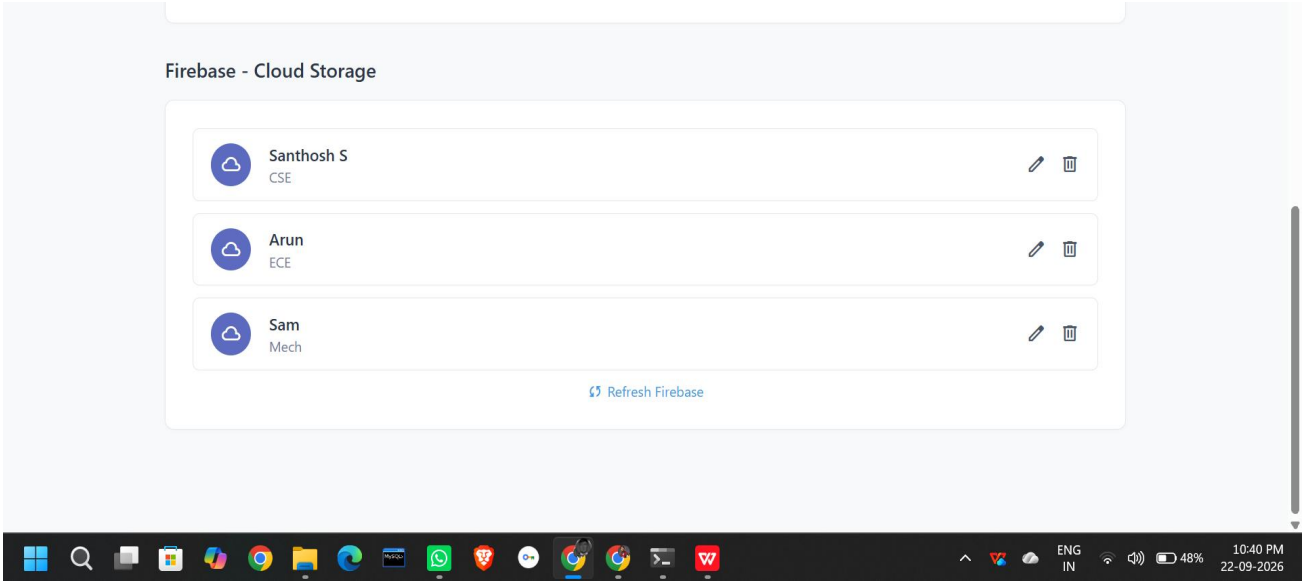
static const FirebaseOptions ios = FirebaseOptions(
apiKey: 'AIzaSyCDi8mRM1HjcyfX_8RfeQ4GHMdkhG99V2s',
appId: '1:291139208558:ios:6bd582928a142796afed7f',
messagingSenderId: '291139208558',
projectId: 'student-firebase-app-b68fd',
storageBucket: 'student-firebase-app-b68fd.firebasestorage.app',
iosBundleId: 'com.example.sqliteApp',
);

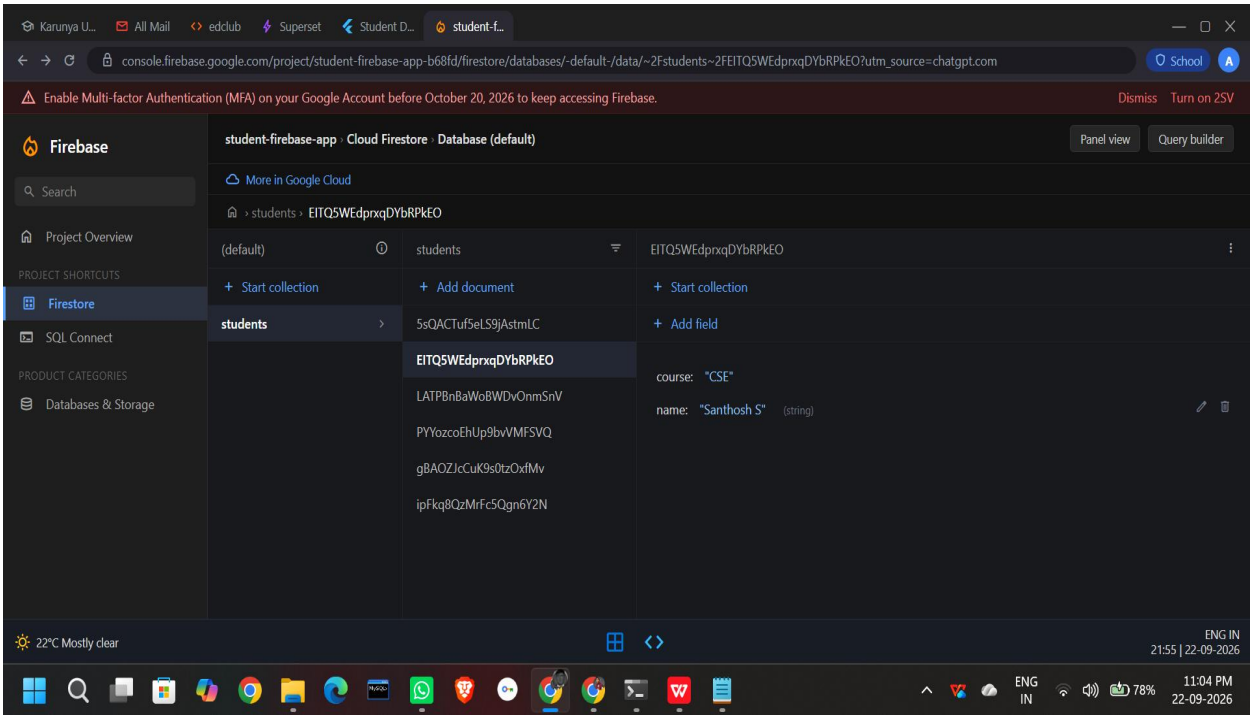
static const FirebaseOptions macos = FirebaseOptions(
apiKey: 'AIzaSyCDi8mRM1HjcyfX_8RfeQ4GHMdkhG99V2s',
appId: '1:291139208558:ios:6bd582928a142796afed7f',
messagingSenderId: '291139208558',
projectId: 'student-firebase-app-b68fd',
storageBucket: 'student-firebase-app-b68fd.firebasestorage.app',
iosBundleId: 'com.example.sqliteApp',
);

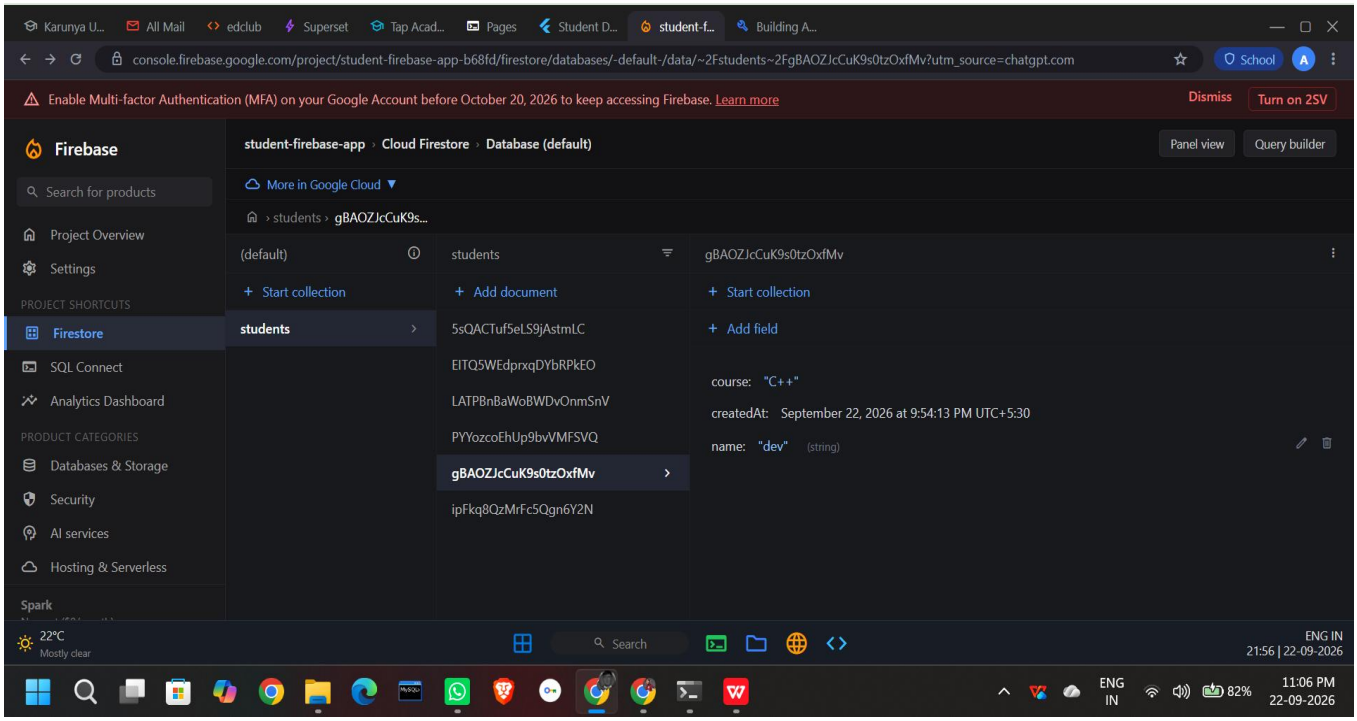
static const FirebaseOptions windows = FirebaseOptions(
apiKey: 'AIzaSyD2_SqRHdwOFdhzLDy5cbXKFpjdPgGuhHg',
appId: '1:291139208558:web:707da545013202cbafed7f',
messagingSenderId: '291139208558',
projectId: 'student-firebase-app-b68fd',
authDomain: 'student-firebase-app-b68fd.firebaseapp.com',
storageBucket: 'student-firebase-app-b68fd.firebasestorage.app',
measurementId: 'G-5TNZX9RJ3M',
);
}
```

Output:


Practicum 8 D
Pages
Student Data
PDF to WORD
Inbox (5,084)
Download hist
Ask Gemini
santhosh444-ai.github.io/FLUTTER-EXP-8/
School
Student Data Persistence
SQLite + Firebase
Web • SQLite WASM + Firebase
Add Student
Student Name
Course
Save to SQLite + Firebase
SQLite - Local Storage
1
Santhosh S
CSE
Refresh SQLite
ENG
IN
49%
10:39 PM
22-09-2026


Firebase - Cloud Storage
Santhosh S
CSE
Arun
ECE
Sam
Mech
Refresh Firebase


Firebase
student-firebase-app › Cloud Firestore › Database (default)
Panel view
Query builder
Project Overview
Firestore
SQL Connect
Databases & Storage
(default)
students
EITQ5WEdprxqDYbRPkEO
Start collection
Add document
Add field
5sQACTuf5eLS9jAstmLC
LATPBnBaWoBWDvOnmSnV
PYYozcoEhUp9bvVMFSVQ
gBAOZJcCuK9s0tzOxfMv
ipFkq8QzMrFc5Qgn6Y2N
course: "CSE"
name: "Santhosh S" (string)


Karunya U...
All Mail
edclub
Superset
Tap Acad...
Pages
Student D...
student-f...
Building A...
console.firebase.google.com/project/student-firebase-app-b68fd/firestore/databases/-default-/data/~2Fstudents~2FgBAOZJcCuK9s0tzOxfMv?utm_source=chatgpt.com
School
Enable Multi-factor Authentication (MFA) on your Google Account before October 20, 2026 to keep accessing Firebase. Learn more
Dismiss
Turn on 2SV
Firebase
student-firebase-app › Cloud Firestore › Database (default)
Panel view
Query builder
Search for products
More in Google Cloud
› students › gBAOZJcCuK9s...
Project Overview
Settings
PROJECT SHORTCUTS
Firestore
SQL Connect
Analytics Dashboard
PRODUCT CATEGORIES
Databases & Storage
Security
AI services
Hosting & Serverless
Spark
(default)
+ Start collection
students
students
+ Add document
5sQACTuf5eLS9jAstmLC
EITQ5WEdprxqDYbRPkEO
LATPBnBaWoBWDvOnmSnV
PYYozcoEhUp9bvVMFSVQ
gBAOZJcCuK9s0tzOxfMv
ipFkq8QzMrFc5Qgn6Y2N
gBAOZJcCuK9s0tzOxfMv
+ Start collection
+ Add field
course: "C++"
createdAt: September 22, 2026 at 9:54:13 PM UTC+5:30
name: "dev" (string)
22°C
Mostly clear
Search
ENG IN
21:56 | 22-09-2026
ENG IN
82%
11:06 PM
22-09-2026

RESULT

Thus, data persistence was successfully implemented in Flutter using SQLite and Firebase. Student records were stored locally using SQLite and remotely using Firebase Cloud Firestore, and the stored data was successfully retrieved and displayed in the application.